



## TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the [document register](#) for the full explanation.

**Status: Documented — this project's strongest evidence. Clause: 5.7 —**  
Software System Testing · **Register:** [Document Register](#)

### What the standard requires (Class C — every sub-clause of 5.7)

| REF   | TITLE                                     | APPLIES AT | OUTPUT                                                                                                                                      |
|-------|-------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 5.7.1 | Establish tests for software requirements | A, B, C    | A set of tests covering all software requirements: input stimuli, expected outcomes, pass/fail criteria, procedures                         |
| 5.7.2 | Use software problem resolution process   | A, B, C    | —                                                                                                                                           |
| 5.7.3 | Retest after changes                      | A, B, C    | —                                                                                                                                           |
| 5.7.4 | Evaluate software system testing          | A, B, C    | Recorded traceability between requirements and tests, plus evidence results meet pass/fail criteria                                         |
| 5.7.5 | Test record contents                      | A, B, C    | Test case reference, result and anomalies, software version tested, hardware/software test configuration, test tools, date, tester identity |

(All five apply to every class — Amendment 1 moved 5.7 from B/C-only to all classes; see the site's own Safety Class Applicability section for why that history matters to how this data is maintained.)

### 5.7.1 — The test set

tests/test\_site.py is this project's system test suite: **606 checks** across 12 groups, run against a real Chrome/Chromium browser driven by Playwright, over a server the script starts itself. Its own docstring states the design directly:

*"It is a black-box test suite. It does not import or inspect the site's JavaScript; it drives the pages the way a person would — clicking, typing, reading what appears — and asserts on the result."*

| GROUP         | WHAT IT COVERS                                                                                                                                                                                                                                     |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| data          | JSON parses; correct counts; no duplicate ids/questions; every correct index in range; all required fields present                                                                                                                                 |
| applicability | Sub-clause mapping is complete; clause-level classes equal the union of sub-clause classes; spot checks against the standard; rendered rows/counts; <b>regression test reintroducing the original Clause 7 error</b>                               |
| deliverables  | Per-class output counts; CSV filename/BOM/CRLF/columns/escaping; print output; §4.1/§4.2 cross-references                                                                                                                                          |
| docs          | Every card's example-document Preview/Download links resolve correctly; every rendered page in this folder loads without a JS error, carries the training-example banner, links back to the right card, has no horizontal overflow, and passes axe |
| learn         | Card rendering; expand/collapse by mouse and keyboard; level toggle; class filters; filter notice; progress tracker; async failure paths (404, malformed JSON, empty list, retry)                                                                  |
| quiz          | Name validation; scoring; timer; full pass/fail runs; certificate; shuffling; prefetch; error states                                                                                                                                               |

| GROUP      | WHAT IT COVERS                                                                                                                                     |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| contact    | Field validation; request contents; success/422/429/404/503/network-failure paths; double-submit guard; honeypot                                   |
| privacy    | Notice content; footer links; provenance section                                                                                                   |
| version    | Version chip; Edition 2 notice; machine-readable review date                                                                                       |
| theme      | Toggle, persistence, no-flash, print stays light                                                                                                   |
| a11y       | axe-core WCAG 2.1 A/AA + best practice, light <b>and</b> dark, across loading/error/mid-quiz/feedback/results states; keyboard and focus behaviour |
| responsive | No horizontal overflow at four widths; iOS auto-zoom prevention; tap target sizes; 200% zoom equivalent                                            |

### 5.7.2 — Problem resolution process used

Every failing check in a run is fed into [13 — Problem Resolution](#)'s anomaly log automatically — see `reconcile_anomaly_log()` in `tests/test_site.py`. This is not a manual step a tester has to remember; it happens on every run, pass or fail.

### 5.7.3 — Retest after changes

The suite is not partitioned by "what changed" — a full run re-executes all 606 checks every time, which is retesting by construction rather than by a documented policy of selecting which tests apply to a given change (the same trade-off noted for regression testing in [07](#)). `--group` exists to run a subset during active development, but a change is not considered verified until the full run passes.

### 5.7.4 — Traceability and evaluation

Traceability from requirement to test exists at **group** granularity — see [03 — Requirements Specification](#), where every REQ-ID cites the test group that verifies it. Evidence that results meet pass/fail criteria is the console report `R.report()`

produces at the end of every run: a pass/fail count and a `FAILURES` section naming exactly which check, in which group, and what it returned instead of the expected value.

### 5.7.5 — Test record contents

| REQUIRED FIELD                       | WHERE IT APPEARS                                                                                                                                                   |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test case reference                  | Each check's label, e.g. <code>"Class A: Clause 7 is visible"</code>                                                                                               |
| Result and anomalies                 | <code>PASS / FAIL</code> per check, printed and counted; failures collected in <code>tests/anomaly_log.csv</code>                                                  |
| Software version tested              | Not currently recorded automatically — see Gaps                                                                                                                    |
| Hardware/software test configuration | Browser name printed at the start of each run ( <code>bundled Chromium</code> or <code>system Google Chrome</code> , with a note to pin the version)               |
| Test tools                           | Playwright version pinned in <code>tests/requirements.txt</code> ; axe-core version pinned by URL                                                                  |
| Date                                 | Not currently stamped in the console output — see Gaps                                                                                                             |
| Tester identity                      | Not applicable in the sense the standard means it (an automated run has no human tester), but the invoking user is identifiable from CI/shell history if run there |

### Gaps

- *\*No persistent test execution record beyond console output and the anomaly log — a full pass/fail history across runs (not just currently-open anomalies) is not retained anywhere. `tests/anomaly_Log.csv` records failures with dates; it does not record that a passing\* run happened on a given date with a given software version, which 5.7.5 technically asks for.*

- **No automatic software-version stamp** on a test run — there is no version file or git-commit stamp captured in the console output or the anomaly log. See [09 — Software Release](#) for the related, larger gap this sits inside.